Guide Développeur

Démonstrateur Computer Vision PRI 2025-2026

Projet Recherche et Innovation
Polytech Tours - Département Informatique

Version 1.0 - Février 2026

Table des matières

1 Introduction **4**
1.1 Objectif du Projet . . . 4
1.2 Public Cible . . . 4

2 Architecture Technique **4**
2.1 Vue d'Ensemble . . . 4
2.2 Flux de Données . . . 5

3 Installation et Configuration **5**
3.1 Prérequis . . . 5
3.1.1 Configuration Matérielle Recommandée . . . 5
3.2 Déploiement Rapide (Production) . . . 6
3.3 Développement Local (Sans Docker) . . . 6
3.3.1 Backend . . . 6
3.3.2 Frontend . . . 6

4 Structure du Code **7**
4.1 Organisation Globale . . . 7
4.2 Backend : Points d'Entrée . . . 8
4.2.1 Fichier Principal : `main.py` . . . 8
4.2.2 Reconnaissance Gestuelle : `gesture_recognizer.py` . . . 8
4.3 Frontend : Composants Clés . . . 8
4.3.1 Structure des Pages . . . 8
4.3.2 Hook Critique : `useCamera.tsx` . . . 9
4.3.3 Communication API : `lib/api.ts` . . . 9

5 Gestion des Branches Git **9**
5.1 Stratégie Actuelle . . . 9
5.2 Workflow Recommandé . . . 9

6 Développement Frontend **9**
6.1 Ajout d'un Nouveau Composant . . . 9
6.2 Modification des Styles . . . 10

7 Développement Backend **10**
7.1 Ajout d'un Nouveau Modèle IA . . . 10
7.2 Optimisation des Performances . . . 10
7.3 Entraînement Dobble : Optimisation Auto des Hyperparamètres avec `tune` . . . 10

8 Déploiement **11**
8.1 Construction des Images Docker . . . 11
8.1.1 Backend . . . 11
8.1.2 Frontend . . . 11
8.2 Configuration Nginx . . . 11
8.3 Mise en Production . . . 12

9 Tests et Validation **12**
9.1 Tests Manuels Recommandés . . . 12
9.2 Validation des Modèles IA . . . 12

10 Dépannage **13**
10.1 Problèmes Courants . . . 13
10.1.1 Erreur "Port 8080 already in use" . . . 13
10.1.2 La webcam ne fonctionne pas . . . 13
10.1.3 Modèle YOLO introuvable . . . 13
10.2 Logs de Debugging . . . 14

11 Bonnes Pratiques **14**
11.1 Conventions de Code . . . 14
11.1.1 Frontend (TypeScript) . . . 14
11.1.2 Backend (Python) . . . 14
11.2 Performance . . . 14
11.3 Sécurité . . . 14

12 Ressources Utiles **14**

13 Contact et Support **15**
13.1 Dépôt GitHub . . . 15
13.2 Auteur Original . . . 15
13.3 Licence . . . 15

1 Introduction

Ce guide est destiné aux développeurs qui souhaitent reprendre, maintenir ou étendre le projet **Démonstrateur Computer Vision**. Il détaille l'architecture, la configuration, les flux de développement et les bonnes pratiques adoptées.
Note : Si vous remarquez des manquements n'hésitez pas à consulter le rapport de PRI.

1.1 Objectif du Projet

Application web de démonstration pédagogique de vision par ordinateur, intégrant :

— **Dobble** : Détection d'objets temps réel (YOLOv8)

— **Dino Run** : Reconnaissance gestuelle (MediaPipe + MLP)

1.2 Public Cible

Développeurs ayant des connaissances en :

— TypeScript/React

— Python/FastAPI

— Docker

— Bases de Machine Learning (lecture de documentation)

2 Architecture Technique

2.1 Vue d'Ensemble

L'application adopte une architecture **client-serveur conteneurisée** :

Composant	**Technologies**
Frontend	React 18 + TypeScript + Vite + TailwindCSS
Backend	FastAPI + Uvicorn + Python 3.11
Modèles IA	YOLOv8s (Ultralytics), MediaPipe Hands, TFLite MLP
Serveur Web	Nginx (reverse proxy)
Orchestration	Docker + Docker Compose

Table 1 – Stack technologique

2.2 Flux de Données

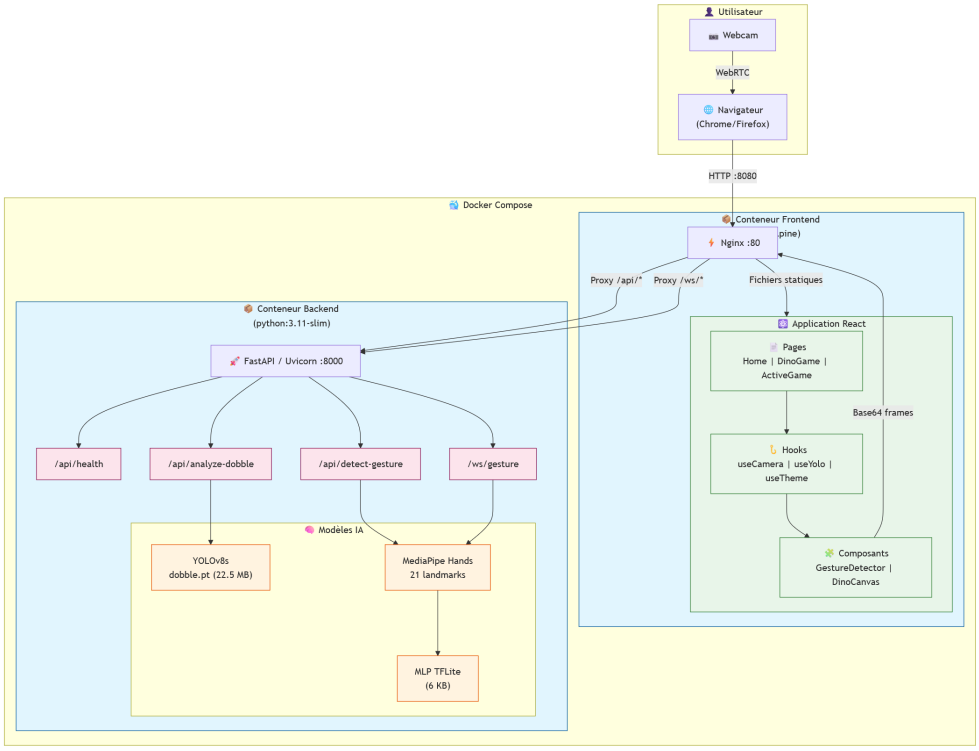

FIGURE 1 – Architecture globale du système : flux de données entre le frontend React, le reverse proxy Nginx, et le backend FastAPI avec les modèles IA.

1. L'utilisateur capture la webcam via le navigateur (WebRTC)
2. Les frames sont encodées en Base64 et envoyées au backend
3. Le backend analyse avec YOLO ou MediaPipe
4. Les résultats sont renvoyés au frontend
5. L'interface affiche les annotations visuelles

3 Installation et Configuration

3.1 Prérequis

— **Docker Desktop** : Version 24.0+ (https://www.docker.com/products/docker-desktop/)
— **Git** : Pour cloner le dépôt
— **Webcam** : Pour tester les fonctionnalités

3.1.1 Configuration Matérielle Recommandée

— RAM : 8 Go minimum (16 Go recommandé)
— Espace disque : 5 Go disponibles
— CPU : 4 cœurs minimum

3.2 Déploiement Rapide (Production)

Listing 1 – Déploiement complet avec Docker

```
# Cloner le depot
git clone https://github.com/Isopope/ComputerVisionPRI25-26.git
cd ComputerVisionPRI25-26

# Lancer les conteneurs
docker-compose up --build

# Acces : http://localhost:8080
```

Note : La première construction prend 40-50 minutes.

3.3 Développement Local (Sans Docker)

Pour un développement plus rapide avec rechargement à chaud :

3.3.1 Backend

Listing 2 – Configuration backend

```
cd backend
python -m venv .venv
.venv\Scripts\activate  # Windows
pip install -r requirements.txt
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

API accessible : `http://localhost:8000/docs`

3.3.2 Frontend

Listing 3 – Configuration frontend

```
cd frontend
npm install
npm run dev
```

Application accessible : `http://localhost:5173`
Important : Modifier `frontend/src/lib/config.ts` pour pointer vers le backend local :

```
export const API_BASE_URL = "http://localhost:8000";
```

4 Structure du Code

4.1 Organisation Globale

```
ComputerVisionPRI25-26/
├── backend/ – Serveur Python FastAPI
│   ├── model/
│   │   ├── keypoint_classifier/
│   │   │   ├── keypoint_classifier.py – Classe d'inférence TFLite
│   │   │   ├── keypoint_classifier.tflite – Modèle MLP (6 KB)
│   │   │   └── keypoint_classifier_label.csv – Labels gestes
│   │   └── point_history_classifier/ – Classifieur trajectoire
│   ├── dobble.pt – Modèle YOLOv8 (22.5 MB)
│   ├── main.py – API REST + WebSocket
│   ├── gesture_recognizer.py – Pipeline MediaPipe
│   ├── Dockerfile – Image Docker multi-stage
│   └── requirements.txt – Dépendances Python
├── frontend/ – Application React/TypeScript
│   ├── src/
│   │   ├── components/
│   │   │   ├── dino/
│   │   │   │   └── EducationalTutorial.tsx – Tutoriel interactif
│   │   │   ├── dobble/
│   │   │   │   └── DobbleExplanationSteps.tsx – Explications YOLO
│   │   │   ├── game-modes/
│   │   │   │   ├── AIPureMode.tsx – Mode IA seule
│   │   │   │   ├── AIvsHumanDobbleMode.tsx – Mode compétitif
│   │   │   │   └── RealtimeDobbleMode.tsx – Détection temps réel
│   │   │   ├── hud/
│   │   │   │   ├── HUDFrame.tsx – Interface Cyberpunk
│   │   │   │   └── SystemStatus.tsx – Indicateurs système
│   │   │   ├── GestureDetector.tsx – Détection gestuelle
│   │   │   ├── DinoGameCanvas.tsx – Canvas Dino Run
│   │   │   ├── BoundingBoxOverlay.tsx – Overlay détections
│   │   │   └── ui/ – Composants Shadcn/UI
│   │   ├── pages/
│   │   │   ├── Home.tsx – Accueil
│   │   │   ├── ModeSelection.tsx – Choix du mode
│   │   │   ├── ActiveGame.tsx – Session de jeu
│   │   │   └── DinoGame.tsx – Page Dino Run
│   │   ├── hooks/
│   │   │   ├── useCamera.tsx – Gestion webcam
│   │   │   ├── useLanguage.tsx – Internationalisation
│   │   │   └── useYolo.tsx – Appels API YOLO
│   │   └── lib/
│   │       ├── api.ts – Client HTTP
│   │       └── config.ts – Configuration
│   ├── Dockerfile – Image Docker Nginx
│   ├── nginx.conf – Configuration reverse proxy
│   └── package.json – Dépendances npm
└── docker-compose.yml – Orchestration conteneurs
```

4.2 Backend : Points d'Entrée

4.2.1 Fichier Principal : `main.py`

Endpoints principaux :

Listing 4 – Endpoints FastAPI

```
@app.get("/api/health")
async def health_check():
    """Check API status"""

@app.post("/api/analyze-dobble")
async def analyze_dobble(request: ImageRequest):
    """Detect Dobble symbols with YOLOv8"""

@app.post("/api/detect-gesture")
async def detect_gesture(request: ImageRequest):
    """Recognize hand gesture with MediaPipe"""

@app.websocket("/ws/gesture")
async def websocket_gesture(websocket: WebSocket):
    """Real-time gesture stream"""
```

Important (usage côté frontend) :

- Le frontend utilise réellement **2 endpoints** :
  - `POST /api/analyze-dobble` (analyse d'image Dobble via HTTP)
  - `WS /ws/gesture` (reconnaissance gestuelle en temps réel via WebSocket)
- `GET /api/health` est un **endpoint de test** (healthcheck) utilisé pour vérifier que l'API répond.
- `POST /api/detect-gesture` correspond à une version **HTTP** de la détection de geste ; elle a été conservée à titre de référence/diagnostic, mais elle était **trop lente** pour un usage de reconnaissance gestuelle **en temps réel** (latence et overhead des requêtes répétées), d'où le choix du WebSocket.

4.2.2 Reconnaissance Gestuelle : `gesture_recognizer.py`

Pipeline MediaPipe :

1. Détection de la main (palm detector)
2. Extraction de 21 landmarks (x, y, z)
3. Normalisation (coordonnées relatives au poignet)
4. Classification par MLP TFLite (4 classes : Open, Close, Pointer, OK)

4.3 Frontend : Composants Clés

4.3.1 Structure des Pages

Page	Rôle
`Home.tsx`	Page d'accueil avec sélection de jeu
`ModeSelection.tsx`	Choix du mode (IA seule, vs Humain, etc.)
`ActiveGame.tsx`	Session de jeu Dobble
`DinoGame.tsx`	Jeu Dino Run avec contrôle gestuel

Table 2 – Pages principales

4.3.2 Hook Critique : `useCamera.tsx`

Gère l'accès webcam via `navigator.mediaDevices.getUserMedia()`.
Fonctionnalités :

— Initialisation du flux vidéo
— Capture de frames (30 FPS)
— Conversion en Base64

4.3.3 Communication API : `lib/api.ts`

Client HTTP pour les appels backend. Utilise `fetch` natif.

5 Gestion des Branches Git

5.1 Stratégie Actuelle

Branche	Usage
`main`	Version stable avec design classique (référence)
`feature/cyberpunk-theme`	Développement actif avec thèmes multiples

Table 3 – Branches Git

5.2 Workflow Recommandé

1. Créer une nouvelle branche pour chaque fonctionnalité :

```
git checkout -b feature/nom-fonctionnalite
```

2. Développer et tester localement
3. Fusionner vers `feature/cyberpunk-theme` après validation
4. Garder `main` pour les versions stables

6 Développement Frontend

6.1 Ajout d'un Nouveau Composant

Exemple : Créer un nouveau mode de jeu pour Dobble

1. Créer le composant dans `src/components/game-modes/`

Listing 5 – Nouveau composant de mode

```
// src/components/game-modes/MyNewMode.tsx
import { useCamera } from "@/hooks/useCamera";

export const MyNewMode = () => {
    const { videoRef, captureFrame } = useCamera();

    return (
        <div>
            <video ref={videoRef} autoPlay />
            {/* Logique du mode */}
        </div>
    );
};
```

2. Enregistrer le mode dans `ModeSelection.tsx`
3. Mettre à jour les traductions dans `hooks/useLanguage.tsx`

6.2 Modification des Styles

Le projet utilise **TailwindCSS** avec un thème Cyberpunk personnalisé.
Fichier de configuration : `tailwind.config.ts`
Exemple : Ajouter une nouvelle couleur

```
theme: {
    extend: {
        colors: {
            'cyberpunk-purple': '#9D4EDD',
        }
    }
}
```

7 Développement Backend

7.1 Ajout d'un Nouveau Modèle IA

Exemple : Intégrer un nouveau jeu utilisant la segmentation d'images

1. Placer le modèle dans `backend/` (ex : `segmentation.pt`)
2. Créer un endpoint dans `main.py` :

```python
from ultralytics import YOLO

segmentation_model = YOLO("segmentation.pt")

@app.post("/api/segment")
async def segment_image(request: ImageRequest):
    # Decode image
    img = decode_base64_image(request.image)

    # Inference
    results = segmentation_model(img)

    # Return masks
    return {"masks": results[0].masks.data.tolist()}
```

3. Ajouter la route au routing Nginx (`frontend/nginx.conf`)

7.2 Optimisation des Performances

Points d'attention :

— Limiter la résolution des images envoyées (640x640 max pour YOLO)
— Utiliser WebSocket pour le streaming temps réel (pas de polling HTTP)
— Activer le cache du modèle (chargé une seule fois au démarrage)

7.3 Entraînement Dobble : Optimisation Auto des Hyperparamètres avec `tune`

La documentation Ultralytics propose la méthode `tune()` permettant d'optimiser automatiquement les hyperparamètres via un algorithme génétique. Cette approche fait évoluer les

paramètres (learning rate, momentum, etc.) afin de maximiser les performances sur un dataset spécifique.

Listing 6 – Recherche d'hyperparamètres avec `tune()`

```
from ultralytics import YOLO

model = YOLO("yolov8s.pt")
model.tune(
    data="data_dobble.yaml",
    epochs=30,
    iterations=100,
    optimizer="AdamW"
)
```

Les meilleurs paramètres sont sauvegardés dans un fichier `best_hyperparameters.yaml`, réutilisable pour les entraînements futurs.

Note : cette méthode n'a pas été utilisée dans le projet, car les hyperparamètres définis de manière empirique (issus de la documentation et d'expérimentations) offraient déjà des résultats satisfaisants. Elle reste cependant une piste d'amélioration pour de futures itérations.

8 Déploiement

8.1 Construction des Images Docker

8.1.1 Backend

Multi-stage build pour optimiser la taille :

Listing 7 – Dockerfile backend (extrait)

```
# Stage 1: Builder
FROM python:3.11-slim AS builder
RUN pip install --no-cache-dir -r requirements.txt

# Stage 2: Runtime
FROM python:3.11-slim
COPY --from=builder /opt/venv /opt/venv
CMD ["uvicorn", "main:app", "--host", "0.0.0.0"]
```

8.1.2 Frontend

Listing 8 – Dockerfile frontend (extrait)

```
FROM node:20-alpine AS builder
WORKDIR /app
RUN npm ci && npm run build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

8.2 Configuration Nginx

Le reverse proxy redirige `/api/*` et `/ws/*` vers le backend :

Listing 9 – nginx.conf (extrait)

```
location /api/ {
    proxy_pass http://backend:8000;
}

location /ws/ {
    proxy_pass http://backend:8000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
}
```

8.3 Mise en Production

Commandes utiles :

Listing 10 – Commandes Docker Compose

```
# Demarrer en arriere-plan
docker-compose up -d

# Consulter les logs
docker-compose logs -f

# Arreter et supprimer les conteneurs
docker-compose down

# Reconstruire sans cache
docker-compose build --no-cache
```

9 Tests et Validation

9.1 Tests Manuels Recommandés

1. **Test Webcam** : Vérifier l'accès caméra sur Chrome/Firefox
2. **Test Dobble** : Positionner 2 cartes et valider la détection
3. **Test Dino** : Effectuer les 4 gestes (Open, Close, Pointer, OK)
4. **Test Multilingue** : Basculer entre français et anglais
5. **Test Thèmes** : Basculer entre Classique et Cyberpunk

9.2 Validation des Modèles IA

Dobble (YOLOv8) :

```
cd backend
python -c "
from ultralytics import YOLO
model = YOLO('dobble.pt')
model.val()
"
```

Dino (MediaPipe + MLP) : Tester manuellement via l'interface.

10 Dépannage

10.1 Problèmes Courants

10.1.1 Erreur "Port 8080 already in use"

Solution : Modifier le port dans `docker-compose.yml` :

```
frontend:
  ports:
    - "8081:80"  # Au lieu de 8080
```

10.1.2 La webcam ne fonctionne pas

Causes possibles :

— Permissions non accordées dans le navigateur

— HTTPS requis (certains navigateurs bloquent WebRTC en HTTP)

Solution : Tester avec `chrome://settings/content/camera`

10.1.3 Modèle YOLO introuvable

Erreur : `FileNotFoundError: dobble.pt`
Solution : Vérifier que `backend/dobble.pt` existe.

10.2 Logs de Debugging

```
# Logs backend uniquement
docker-compose logs backend

# Logs frontend uniquement
docker-compose logs frontend

# Logs en temps reel
docker-compose logs -f --tail=50
```

11 Bonnes Pratiques

11.1 Conventions de Code

11.1.1 Frontend (TypeScript)

- Composants en PascalCase : `MyComponent.tsx`
- Hooks personnalisés préfixés `use` : `useCamera.tsx`
- Props typées avec TypeScript

11.1.2 Backend (Python)

- Respect PEP 8
- Type hints pour les fonctions :

```
def process_image(img: np.ndarray) -> dict:
    ...
```

11.2 Performance

- Limiter la taille des images Base64 (compression JPEG 80%)
- Utiliser WebSocket pour éviter la latence HTTP
- Charger les modèles IA au démarrage (pas à chaque requête)

11.3 Sécurité

- **Pas de secrets en dur** : Utiliser des variables d'environnement si besoin
- **Validation des entrées** : Vérifier le format Base64 côté backend
- **CORS** : Configurer strictement les origines autorisées

12 Ressources Utiles

- **Documentation YOLOv8** : https://docs.ultralytics.com/
- **MediaPipe Hands** : https://developers.google.com/mediapipe/solutions/vision/hand_landmarker
- **FastAPI** : https://fastapi.tiangolo.com/
- **React** : https://react.dev/
- **Mediapipe + MLP** : https://github.com/kinivi/hand-gesture-recognition-mediapipe

13 Contact et Support

13.1 Dépôt GitHub

https://github.com/Isopope/ComputerVisionPRI25-26

13.2 Auteur Original

— **Nom** : N'Kouba Humbert

— **Promo** : DI5 - Polytech Tours (2023-2026)

— **Encadrant** : M. Frédéric Rayar

13.3 Licence

Projet académique, libre d'utilisation pour des fins éducatives et de recherche.